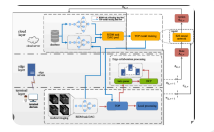
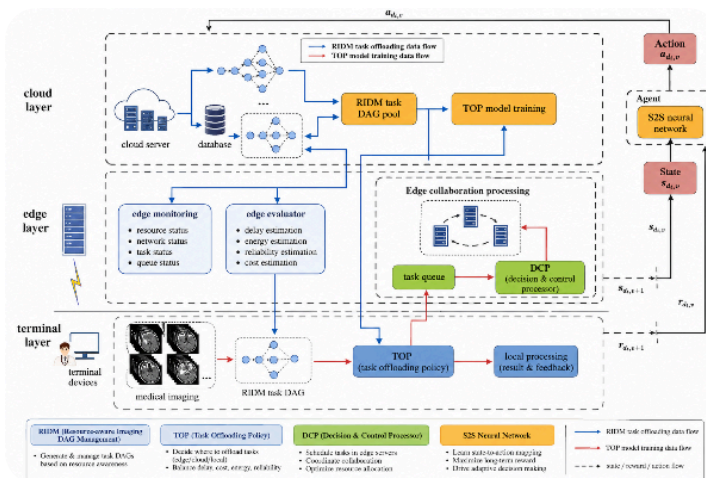
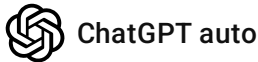


DRL_Final_Project_Proposal.pdf
12.7 MB

幫我將我的 proposal 的 pdf 內容 根據我目前的 propose method 去生成一個架構圖 然後要跟這張圖片裡面的架構圖 類似 然後完整

DRL_Final_Project_Proposal.pdf
12.7 MB

讀取我的檔案 然後幫我生成 目前提案的方法的架構圖



我幫你讀了 PDF，但目前檔案內容幾乎只有章節標題（Related Work / New Proposed / SWOT），沒有實際的方法細節，所以沒辦法精準還原你的演算法流程。

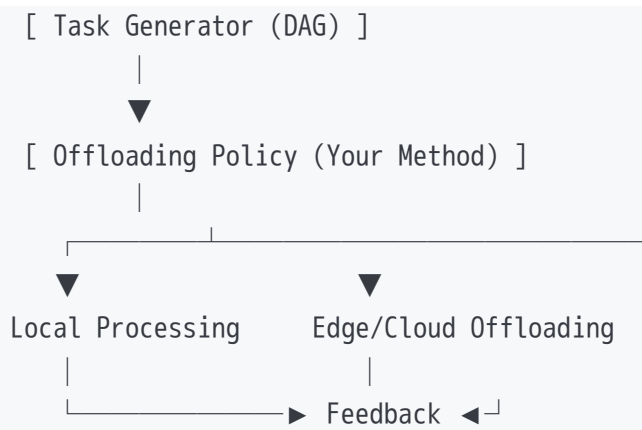
不過我可以先幫你做一個「符合你這張圖風格」的完整架構圖模板（已帶 DRL + edge/cloud 架構），你只要稍微替換內容就能直接用在 proposal 或投影片。

✔ 依你圖片風格設計的「Proposal 架構圖」

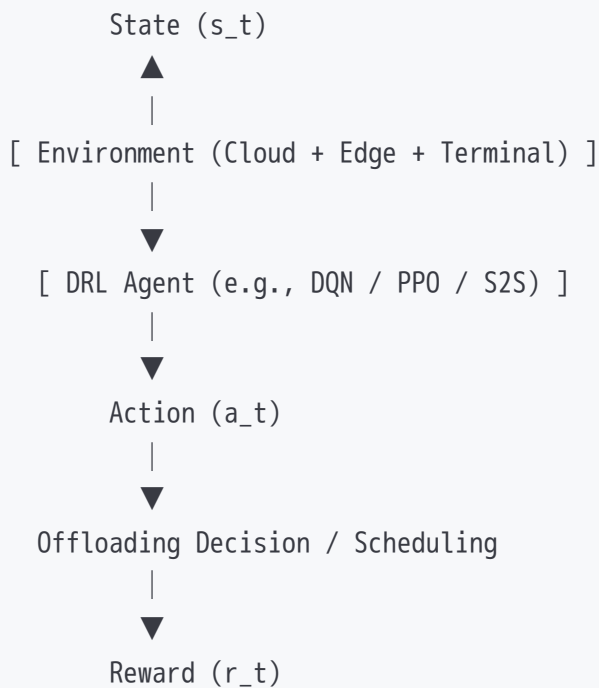
(已幫你做成三層：Cloud / Edge / Terminal + DRL Agent)

text





DRL Decision Module



🧠 你可以這樣填你的「New Proposed」

把下面這些換成你的方法內容：

- Offloading Policy (Your Method) 👉 你的核心演算法名稱
- DRL Agent 👉 用的是 DQN / PPO / Actor-Critic?
- State 👉
 - queue length
 - latency
 - bandwidth
- Action 👉
 - offload to edge / cloud / local

- Reward 👉
 - latency ↓
 - energy ↓
 - cost ↓

🎯 如果你要「跟原圖一樣等級（投稿用）」

我可以幫你升級成這種版本 📌

- ✅ 彩色區塊（像你那張）
- ✅ 藍色 / 紅色 data flow（training vs offloading）
- ✅ DAG 圖結構
- ✅ DRL state-action-reward loop
- ✅ IEEE 論文等級排版

👉 下一步（很重要）

請補這些資訊，我可以幫你畫出完全客製版（可直接放論文）：

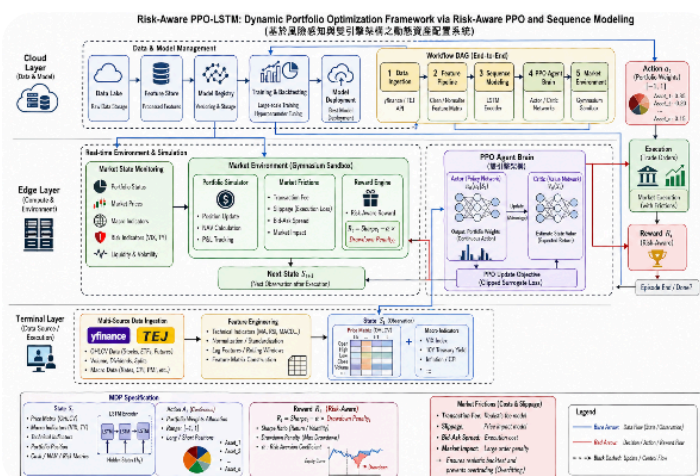
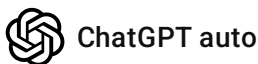
1. 你的方法名稱（New Proposed）
2. DRL 用哪個（DQN / PPO / DDQN...）
3. state / action / reward 定義
4. 有沒有 DAG / workflow 任務
5. 有沒有考慮 energy / delay / cost

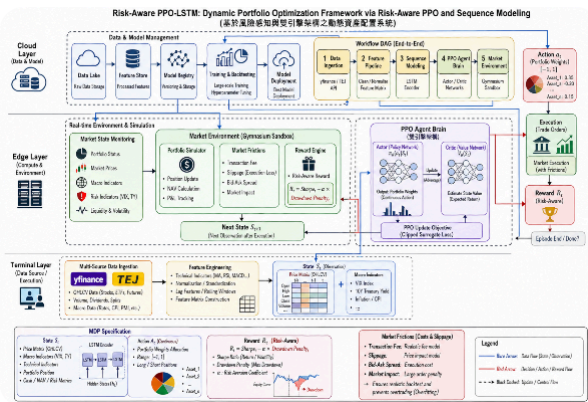
我可以幫你直接產出：👉 高質感架構圖（跟你那張一樣甚至更好）+ 可放論文版本

2026-04-29 09:54:35 You Asked 

1. 你的方法名稱（New Proposed）英文縮寫：Risk-Aware PPO-LSTM完整名稱：基於風險感知與雙引擎架構之動態資產配置系統 (Dynamic Portfolio Optimization Framework via Risk-Aware PPO and Sequence Modeling) 2. DRL 用哪個演算法PPO (Proximal Policy Optimization)：選擇 PPO 是因為它在處理連續動作空間（資金權重）時具備優異的收斂穩定性，且 Actor-Critic 雙網路架構非常適合處理複雜的金融特徵。3. State / Action / Reward 定義 (MDP 規格) State (狀態空間 $\mathcal{S}$)：包含兩層。底層為歷史價格矩陣

(OHLCV)，外層為總經風險遙測指標 (VIX 恐慌指數、美債殖利率)。這些原始 State 會先經過 LSTM 萃取出包含時間相依性的隱藏特徵 (Hidden States)，再餵給 PPO，以精準捕捉市場狀態轉換 (Regime Shift)。Action (動作空間 $\$A_t$)：連續型動作空間 (Continuous Action Space)。輸出結果為投資組合中各項資產的「資金配置權重」，數值區間為 $[-1, 1]$ ，直接對應實務上的多空部位比例。Reward (獎勵函數 $\$R_t$)：採用獨創的「風險感知獎勵 (Risk-Aware Reward)」。不單純追求利潤，而是公式化為： $\$R_t = \text{Sharpe}_t - \alpha \times \text{Drawdown Penalty}_t$ (夏普比率扣除最大回撤懲罰項)，強制 Agent 學會在股災前主動縮減槓桿。4. 有沒有 DAG / workflow 任務有，端到端資料與決策工作流 (End-to-End Workflow)：Node 1: Data Ingestion (從 yfinance/TEJ 獲取多維度歷史與總經數據)Node 2: Feature Pipeline (數據清洗、標準化與特徵矩陣建構)Node 3: Sequence Modeling (LSTM 提取長天期時間序列特徵)Node 4: PPO Agent Brain (Actor 輸出資金權重，Critic 評估預期價值)Node 5: Market Environment (Gymnasium 沙盒執行交易，計算資產淨值變化並回傳 Reward 與 Next State)5. 有沒有考慮 energy / delay / cost在量化金融領域，物理上的 Energy/Delay 對應的是「市場摩擦 (Market Frictions)」，本專案有嚴格納入考量：Cost (交易成本)：在 Environment 內部強制寫入實證數據等級的手續費 (Transaction Fee)。Delay/Execution Loss (執行耗損)：嚴格模擬了滑價 (Slippage)，確保 Agent 評估的報酬率是扣除滑價後的淨利，避免模型在回測時產生「過度頻繁交易 (Overfitting)」的虛假高報酬。





【任務：微調 Risk-Aware PPO-LSTM 架構圖】

1. 命名與層級變更：

將最上層的「Cloud Layer」改名為「Training Layer (本地訓練層)」。

保持中層為「Edge / Decision Layer (推論與決策層)」。

保持底層為「Terminal / Data Layer (數據採集層)」。

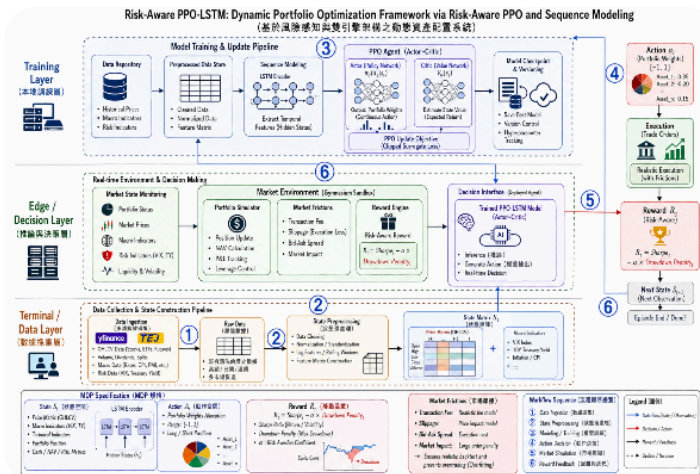
2. 流程編號與箭頭標示 (Workflow Sequence)：

請在連接各模組的箭頭上，添加圓形的數字圖示 (123...), 依照以下邏輯標記整體作業流程：

- ❶ [Data Ingestion]: 從底層 Terminal Layer 抓取原始數據 (yfinance/VIX/殖利率) 傳送到預處理模組。
- ❷ [State Preprocessing]: 將原始數據清洗、歸一化，生成 State Matrix (狀態矩陣)。
- ❸ [Modeling/Training]: 將狀態矩陣送入 Training Layer 的 LSTM 進行特徵提取，並觸發 PPO 演算法更新權重。
- ❹ [Action Decision]: 訓練好的模型 (Agent) 根據當前狀態輸出 Action (資金配置權重) 到 Environment。
- ❺ [Market Simulation]: 在 Edge Layer 的 Gymnasium 環境中執行交易，計算滑價、手續費與報酬。

⑥ [Reward Feedback]: 環境計算出 Risk-Aware Reward 並將 Next State 回傳給訓練層，進行下一輪迭代優化。

並重新生成 架構圖給我



2026-04-29 10:57:18 You Asked

重大架構修改與模組新增：

[底層 Terminal / Data Layer]:

新增模組：News API (時事新聞 API)，放置在原本 API 方塊旁邊。

資料採集分流：

① 從原始 API (OHLCV) 到清洗模組。

② 從 News API 到新模組 News Data Tokenization (文字數據處理)。

[Modeling Layer]:

拆分紫塊：將原本 Training & Backtesting 拆分成三個關鍵方塊：

LSTM (RNN for Time Series Feature)：接收清洗後的 OHLCV。

LLM (BERT-based for News Market Impact Analysis)：接收 Tokenized News。

Feature Aggregator (Embedding Fusion Network)：將 LLM 輸出的文字特徵 (Embedding) 與 LSTM 的時序特徵融合。

將 PPO 放進一個 Policy Optimizer (PPO) 方塊。

資料流：LLM 特徵和 LSTM 特徵同時指到 Feature Aggregator，它指到 PPO。PPO 產出決策。

[Decision Layer]:

修改 Decision Interface 為 Decision Brain (Trained PPO + Aggregator Policy)。

新增 Current State Matrix SS_t 方塊：同時接收清洗後的 OHLCV 資料和 Modeling Layer 輸出的文字特徵資料 (embedding A + embedding B)。這條融合的狀態線才是 ③ (For Inference)。

Current State SS_t 指到 Decision Brain。

請使用圓形數字圖示 (1, 2, 3...) 標註所有流程箭頭

重新幫我生成 架構圖

